

CLASSPULSE

The Classroom That Teaches Back

Product Requirements Document & Technical Architecture

Version 1.0 — March 2026

Tech Stack: React · FastAPI · PostgreSQL · Redis · Docker · Nginx · Celery · AWS/Render

Table of Contents

1. Product Vision & Problem Statement
2. User Personas
3. Core Concept & Product Philosophy
4. Feature Specification (MVP)
5. Complete User Workflows
6. Database Schema Design
7. API Endpoint Design
8. System Architecture
9. Real-Time Architecture (WebSocket)
10. Background Jobs (Celery)
11. LLM Integration Design
12. DevOps & Deployment
13. Non-Functional Requirements
14. MVP Scope & Phased Roadmap

1. Product Vision & Problem Statement

1.1 The Problem

Google Classroom dominates education but operates as a file dump — students submit work into a void and wait days or weeks for feedback. The fundamental feedback loop of learning (attempt → feedback → improve → reattempt) is completely broken.

Specific Pain Points

- **No draft feedback:** Students submit their first and only attempt. There is no mechanism to get feedback on a work-in-progress before the deadline.
- **No unified deadline view:** Students enrolled in 5-6 courses must open each classroom separately to check deadlines. There is no cross-course calendar.
- **Grading is manual chaos:** Teachers grade in spreadsheets or one-by-one in the UI, copy-pasting the same feedback 60 times for the same common mistakes.
- **No learning analytics:** Neither students nor teachers can see performance trends over time. A student cannot answer: Am I improving in this subject?
- **No rubric-linked grading:** Google Classroom has basic rubrics but no way to click a rubric criterion and have it auto-calculate the score with pre-written feedback.
- **Indian context (300+ student batches):** Large class sizes make individual feedback physically impossible. Teachers need force-multiplier tools.

1.2 The Vision

ClassPulse is a classroom management platform where the assignment submission process itself becomes a learning experience. The core innovation: students can submit drafts before the final deadline and receive instant, rubric-aligned AI feedback — giving them a chance to improve before the final submission. Teachers define the rubric once, and the system generates specific, actionable feedback for every student automatically.

1.3 One-Line Pitch

"Google Classroom, but it actually helps you learn — not just submit."

2. User Personas

2.1 Student — Priya (B.Tech CSE, 3rd Year)

- **Enrolled in:** 5 courses this semester
- **Pain:** Misses deadlines because she checks each Google Classroom separately. Gets 4/10 on assignments with no feedback on what went wrong. Wants to improve but has no signal on what to fix.
- **Wants:** One dashboard showing all deadlines across all courses. A way to check if her answer is good enough before submitting. Grade trends to track improvement.

2.2 Teacher — Prof. Ramesh (Dept. of CSE, 15 years exp.)

- **Teaches:** 3 courses, 180 students total across sections
- **Pain:** Spends 2 full weekends per month grading assignments. Copy-pastes the same feedback for common mistakes. Has no way to identify struggling students early. Rubrics exist on paper but not in the grading tool.
- **Wants:** Click-to-grade using rubric criteria. AI that drafts feedback he can approve/edit. A dashboard showing class-wide patterns (which topics students struggle with).

2.3 Admin / HOD — Dr. Meena (Head of Department)

- **Oversees:** 12 faculty, 600 students
- **Pain:** No visibility into whether assignments are being given, graded on time, or if students are falling behind. NAAC accreditation requires proof of continuous assessment.
- **Wants:** Department-wide dashboard showing assignment frequency, grading turnaround, and at-risk students.

3. Core Concept & Product Philosophy

3.1 The Draft-Feedback Loop (Core Innovation)

This is what makes ClassPulse fundamentally different from Google Classroom, Moodle, or any existing LMS:

1. Teacher creates an assignment with a rubric (criteria + marks + description of what good/bad looks like).
2. Student submits a draft at any time before the deadline.
3. System instantly evaluates the draft against the rubric using LLM and provides specific, criterion-level feedback. Example: "Criterion 2 (Normalization): You explained 1NF and 2NF but missed 3NF and BCNF. Add these with examples to improve from 3/10 to 7/10."
4. Student improves their work based on feedback and submits again (unlimited drafts, or teacher-configured limit).
5. Student makes a final submission before deadline.
6. Teacher sees the final submission + all draft history + AI-suggested grades. Teacher approves, adjusts, or overrides.

This transforms the assignment from a one-shot exam into an iterative learning process. The AI is not grading — the teacher is. The AI is coaching the student before the teacher even sees the work.

3.2 Design Principles

- **Simple over feature-rich:** A teacher should be able to create a classroom and post an assignment in under 3 minutes.
- **Feedback is the product:** Every feature should accelerate or improve the feedback loop.
- **Teacher stays in control:** AI suggests, teacher decides. No grade is ever auto-assigned without teacher approval.
- **Works for 300 students:** Every feature must work at Indian batch sizes. Bulk operations, not one-by-one.

4. Feature Specification (MVP)

4.1 Classroom Management

Feature	Description
Create Classroom	Teacher creates a classroom with name, subject code, section, semester. Gets a unique 6-char join code.
Join via Code	Students enter 6-char code to join. No approval needed (configurable).
Role System	Three roles: Student, Teacher, Admin. Teacher can add co-teachers. Admin sees all classrooms in department.
Classroom Settings	Toggle: auto-approve joins, allow late submissions, max draft count, enable AI feedback.

4.2 Assignment System

Feature	Description
Create Assignment	Title, description (markdown), deadline (date + time), total marks, assignment type (text/file/both).
Rubric Builder	Add criteria: name, max marks, description of what excellent/good/average/poor looks like. These criteria power the AI feedback.
Draft Submission	Student submits work and gets instant AI feedback against rubric. Can resubmit up to N times (configurable).
Final Submission	Explicit "Submit Final" action. Locks the submission. Shows diff from last draft.
Late Submission	Configurable: block after deadline, allow with penalty (auto-calculated), or allow with flag.

4.3 AI Feedback Engine

Feature	Description
Draft Feedback	LLM reads rubric criteria + student submission. Generates per-criterion feedback with estimated score and specific improvement suggestions.
Feedback Tone	Constructive and encouraging. Never says "wrong" — says "to improve from 5/10 to 8/10, add X."
Teacher Override	Teacher can edit any AI-generated feedback before it reaches student. Can also disable AI for specific assignments.
Similarity Detection	Cosine similarity on text submissions between classmates. Flags pairs above threshold. No ML — uses TF-IDF vectors.

4.4 Grading System

Feature	Description
Rubric Click-Grading	Teacher sees rubric criteria as clickable cards. Click "Excellent" / "Good" / "Average" / "Poor" per criterion. Score auto-calculates.
AI Grade Suggestion	System pre-fills suggested grades based on AI analysis. Teacher approves or adjusts with one click.
Bulk Feedback	For common mistakes, teacher writes feedback once and applies to multiple students.
Grade Release	Grades are not visible until teacher explicitly releases them. Batch release for entire assignment.

4.5 Student Dashboard

Feature	Description
Unified Deadline Calendar	Single view showing ALL deadlines across ALL enrolled classrooms. Color-coded by urgency (red = <24h, orange = <3 days, green = >3 days).
Grade Trends	Line chart showing grade trajectory per subject over time. Overall GPA trend.
Draft History	See all drafts for an assignment with AI feedback at each stage. Track improvement.
Notification Center	Real-time notifications: new assignment posted, deadline reminder (24h, 1h), grade released, feedback received.

4.6 Teacher Dashboard

Feature	Description
Submission Tracker	Live count: "45/60 submitted" with list of who hasn't submitted yet. Real-time via WebSocket.
Class Analytics	Average score per assignment, score distribution histogram, topic-wise performance (if rubric criteria map to topics).
At-Risk Students	Students whose rolling average drops below configurable threshold. Flagged automatically.
Grading Queue	Prioritized list: ungraded assignments sorted by deadline proximity. One-click navigation to grading view.

5. Complete User Workflows

5.1 Teacher: Create Classroom & Post Assignment

7. Teacher signs up / logs in (email + password, JWT-based auth).
8. Clicks "Create Classroom" → enters name, subject code, section, semester.
9. System generates 6-char join code (e.g., "CSE4A7"). Teacher shares with students.
10. Teacher clicks "New Assignment" → enters title, description (markdown editor), deadline, total marks.
11. Teacher builds rubric: adds 3-5 criteria, each with name, max marks, and level descriptors (Excellent / Good / Average / Poor with descriptions).
12. Teacher sets config: max drafts allowed (default: 3), late submission policy, AI feedback on/off.
13. Clicks "Publish" → all enrolled students receive real-time WebSocket notification + optional email.

5.2 Student: Submit Draft & Get Feedback

14. Student sees new assignment notification on dashboard + in unified calendar.
15. Opens assignment → reads description and rubric criteria.
16. Writes answer in rich text editor (or uploads file, depending on assignment type).
17. Clicks "Submit Draft" → system queues LLM feedback job via Celery.
18. Within 15-30 seconds, feedback appears: per-criterion assessment with estimated score and specific improvement suggestions.
19. Student reads feedback, improves answer, submits another draft (if drafts remaining > 0).
20. When satisfied, clicks "Submit Final" → submission is locked. Timestamp recorded. Teacher is notified.

5.3 Teacher: Grade Submissions

21. Teacher opens assignment → sees submission tracker (52/60 submitted).
22. Enters grading view → sees list of submissions with AI-suggested grades pre-filled.
23. For each submission: sees student's final answer + rubric criteria as clickable cards.
24. Clicks quality level per criterion (Excellent/Good/Average/Poor). Score auto-calculates.
25. Reviews AI-suggested feedback. Edits if needed. Clicks "Save Grade".
26. For common mistakes: writes feedback once, selects 15 students with same issue, applies to all.
27. When done, clicks "Release Grades" → all students receive notification + can view feedback.

5.4 Student: View Results & Track Progress

28. Student receives "Grade Released" notification.
29. Opens assignment → sees total score, per-criterion breakdown, and teacher feedback.
30. Can view draft history: "Draft 1: 4/10 → Draft 2: 6/10 → Final: 8/10" showing improvement trajectory.
31. Navigates to grade trends → sees line chart of performance across all assignments in the course.
32. Sees cross-course view: all subjects, overall trend, which subjects are improving vs declining.

6. Database Schema Design

PostgreSQL with proper relational normalization. All tables include created_at, updated_at timestamps with timezone.

6.1 Core Tables

users

Column	Type	Notes
id	UUID (PK)	Primary key, auto-generated
email	VARCHAR(255)	Unique, indexed. Login credential.
password_hash	VARCHAR(255)	bcrypt hashed password
full_name	VARCHAR(100)	Display name
role	ENUM	student / teacher / admin
avatar_url	VARCHAR(500)	Optional profile picture URL

classrooms

Column	Type	Notes
id	UUID (PK)	
name	VARCHAR(100)	e.g., "Database Management Systems"
subject_code	VARCHAR(20)	e.g., "18CS53"
section	VARCHAR(10)	e.g., "4A"
semester	VARCHAR(10)	e.g., "5th Sem"
join_code	CHAR(6)	Unique, indexed. Auto-generated.
created_by	UUID (FK)	References users.id (teacher)
settings	JSONB	{ max_drafts: 3, late_policy: "penalty", ai_feedback: true }

enrollments

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK)	References users.id
classroom_id	UUID (FK)	References classrooms.id
role	ENUM	student / co_teacher
joined_at	TIMESTAMPTZ	When student joined

Unique constraint on (user_id, classroom_id). Indexed on classroom_id for fast lookups.

assignments

Column	Type	Notes
id	UUID (PK)	
classroom_id	UUID (FK)	References classrooms.id
title	VARCHAR(200)	Assignment title
description	TEXT	Markdown content
deadline	TIMESTAMPTZ	Submission deadline
total_marks	INTEGER	Maximum score
submission_type	ENUM	text / file / both
max_drafts	INTEGER	Override classroom default
late_policy	ENUM	block / penalty / allow
penalty_per_day	DECIMAL	Marks deducted per day late
is_published	BOOLEAN	Draft vs published
created_by	UUID (FK)	Teacher who created

rubric_criteria

Column	Type	Notes
id	UUID (PK)	
assignment_id	UUID (FK)	References assignments.id
name	VARCHAR(100)	e.g., "Normalization"
max_marks	INTEGER	Marks for this criterion
order_index	INTEGER	Display order
levels	JSONB	{ excellent: "...", good: "...", average: "...", poor: "..." }

submissions

Column	Type	Notes
id	UUID (PK)	
assignment_id	UUID (FK)	References assignments.id
student_id	UUID (FK)	References users.id
content	TEXT	Text submission content
file_url	VARCHAR(500)	Uploaded file path (if file type)
is_final	BOOLEAN	True = final, False = draft
draft_number	INTEGER	1, 2, 3...
is_late	BOOLEAN	Auto-calculated from deadline

submitted_at	TIMESTAMPTZ	Immutable timestamp
--------------	-------------	---------------------

Index on (assignment_id, student_id, is_final). Unique constraint on (assignment_id, student_id) where is_final = true.

ai_feedback

Column	Type	Notes
id	UUID (PK)	
submission_id	UUID (FK)	References submissions.id
criterion_id	UUID (FK)	References rubric_criteria.id
estimated_score	INTEGER	AI suggested score for this criterion
feedback_text	TEXT	Specific improvement suggestion
generated_at	TIMESTAMPTZ	When AI generated this

grades

Column	Type	Notes
id	UUID (PK)	
submission_id	UUID (FK)	References submissions.id (final only)
criterion_id	UUID (FK)	References rubric_criteria.id
score	INTEGER	Teacher-assigned score
level	VARCHAR(20)	excellent / good / average / poor
feedback	TEXT	Teacher feedback for this criterion
graded_by	UUID (FK)	Teacher who graded
is_released	BOOLEAN	Visible to student?

notifications

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK)	Recipient
type	ENUM	assignment_posted / deadline_reminder / grade_released / feedback_ready
payload	JSONB	{ classroom_id, assignment_id, message }
is_read	BOOLEAN	Default false

7. API Endpoint Design

RESTful API with FastAPI. All endpoints return JSON. Authentication via JWT Bearer token in Authorization header. Prefix: /api/v1

7.1 Auth Endpoints

Method	Endpoint	Description
POST	/auth/register	Register with email, password, full_name, role
POST	/auth/login	Returns access_token (15min) + refresh_token (7 days)
POST	/auth/refresh	Exchange refresh_token for new access_token
GET	/auth/me	Get current user profile

7.2 Classroom Endpoints

Method	Endpoint	Description
POST	/classrooms	Create classroom (teacher only)
GET	/classrooms	List user's classrooms (enrolled + created)
GET	/classrooms/:id	Get classroom details + members
POST	/classrooms/join	Join via 6-char code
PUT	/classrooms/:id/settings	Update classroom settings
GET	/classrooms/:id/analytics	Class-wide performance analytics

7.3 Assignment Endpoints

Method	Endpoint	Description
POST	/classrooms/:id/assignments	Create assignment with rubric
GET	/classrooms/:id/assignments	List assignments in classroom
GET	/assignments/:id	Get assignment details + rubric
PUT	/assignments/:id	Update assignment (before any submissions)
POST	/assignments/:id/publish	Publish assignment + notify students
GET	/assignments/:id/submissions	Teacher: list all submissions

7.4 Submission Endpoints

Method	Endpoint	Description
POST	/assignments/:id/submit-draft	Submit draft, triggers AI feedback job

POST	/assignments/:id/submit-final	Submit final (locks submission)
GET	/assignments/:id/my-submissions	Student: get all drafts + final + feedback
GET	/submissions/:id/feedback	Get AI feedback for a specific draft

7.5 Grading Endpoints

Method	Endpoint	Description
POST	/submissions/:id/grade	Teacher: submit criterion-level grades
POST	/assignments/:id/release-grades	Release all grades for an assignment
POST	/assignments/:id/bulk-feedback	Apply same feedback to multiple students

7.6 Dashboard Endpoints

Method	Endpoint	Description
GET	/dashboard/deadlines	Student: all upcoming deadlines across classrooms
GET	/dashboard/grade-trends	Student: grade trends per classroom + overall
GET	/dashboard/grading-queue	Teacher: ungraded assignments sorted by urgency
GET	/dashboard/at-risk	Teacher: students below performance threshold

7.7 WebSocket Endpoint

Protocol	Endpoint	Description
WS	/ws/notifications	Per-user notification channel. Sends: assignment_posted, deadline_reminder, grade_released, feedback_ready, submission_count_update.

8. System Architecture

8.1 Service Overview

Six Docker containers orchestrated via Docker Compose:

- **api (FastAPI):** Main application server. Handles HTTP requests, WebSocket connections, and business logic. Runs on Uvicorn with 4 workers.
- **worker (Celery):** Background task processor. Handles AI feedback generation, email dispatch, similarity detection, PDF report generation.
- **beat (Celery Beat):** Periodic task scheduler. Triggers deadline reminders (every hour), at-risk student detection (daily), stale notification cleanup (weekly).
- **db (PostgreSQL 16):** Primary data store. All relational data. Connection pooling via asyncpg.
- **redis (Redis 7):** Multi-purpose: Celery message broker, WebSocket pub/sub channels, session cache, rate limiting (sliding window), submission count cache.
- **nginx:** Reverse proxy. Handles SSL termination, WebSocket upgrade (Upgrade: websocket header), static file serving, file upload size limits (max 10MB), rate limiting on /auth endpoints.

8.2 Request Flow

33. Client (React SPA) sends request to Nginx on port 80/443.
34. Nginx routes /api/* to FastAPI (port 8000), /ws/* to WebSocket handler, and /* to React static files.
35. FastAPI validates JWT, executes business logic, reads/writes PostgreSQL.
36. For async operations (AI feedback, emails), FastAPI pushes task to Redis queue.
37. Celery worker picks up task, processes it, writes result to PostgreSQL.
38. Worker publishes notification to Redis pub/sub channel.
39. FastAPI WebSocket handler picks up pub/sub message and pushes to connected client.

9. Real-Time Architecture (WebSocket)

9.1 Connection Lifecycle

40. Client connects to /ws/notifications with JWT token as query param.
41. Server validates JWT, extracts user_id.
42. Server subscribes to Redis pub/sub channel: user:{user_id}:notifications
43. Any service (API server, Celery worker) can publish to this channel.
44. Server forwards Redis messages to WebSocket client.
45. Client implements heartbeat ping every 30 seconds. Server closes stale connections after 60 seconds of no ping.

9.2 Event Types

Event	Payload
assignment_posted	{ classroom_id, assignment_id, title, deadline }
deadline_reminder	{ assignment_id, title, hours_remaining }
feedback_ready	{ submission_id, assignment_title }
grade_released	{ assignment_id, title, score, total }
submission_count	{ assignment_id, count, total_students } (teacher only)

9.3 Redis Channel Design

- **Per-user channel:** user:{user_id}:notifications — for personal notifications (grades, feedback).
- **Per-classroom channel:** classroom:{classroom_id}:events — for classroom-wide events (new assignment). Server subscribes to all classrooms the user is enrolled in.

10. Background Jobs (Celery)

10.1 Task Definitions

Task	Trigger	Description
generate_ai_feedback	On draft submit	Sends submission + rubric to LLM API. Stores per-criterion feedback in ai_feedback table. Publishes feedback_ready notification via Redis.
send_deadline_reminders	Celery Beat (hourly)	Queries assignments with deadline in next 24h and 1h. Creates notifications for students who haven't submitted yet.
detect_similarity	On final submit	Computes TF-IDF vectors for all final submissions in an assignment. Flags pairs with cosine similarity > 0.85. Stores in similarity_flags table.
detect_at_risk_students	Celery Beat (daily)	Queries students with rolling average < threshold. Creates at-risk flags for teacher dashboard.
generate_report_pdf	On demand	Generates PDF report card for a student or class analytics for a teacher.
cleanup_stale_notifications	Celery Beat (weekly)	Deletes read notifications older than 30 days.

11. LLM Integration Design

11.1 Prompt Architecture

The LLM receives a structured prompt containing three sections: the rubric criteria with level descriptors, the student's submission text, and an instruction to evaluate each criterion independently.

System Prompt

"You are an academic feedback assistant. You evaluate student submissions against a rubric. For each criterion, provide: (1) an estimated score, (2) what the student did well, (3) specific improvements needed with examples. Your tone is encouraging and constructive. Never say 'wrong' — say 'to improve, add X'. Output valid JSON."

Response Format (JSON)

The LLM returns a JSON array with one object per rubric criterion. Each object contains: `criterion_id`, `estimated_score`, `strengths` (string), `improvements` (string), and `suggested_level` (excellent/good/average/poor).

11.2 Cost & Rate Limiting

- **Model:** GPT-4o-mini or Gemini 1.5 Flash (cost-effective for high volume).
- **Rate limit:** Redis sliding window — max 5 LLM calls per student per hour. Prevents abuse.
- **Caching:** If a student resubmits identical content, return cached feedback instead of calling LLM again. Cache key: `SHA256(submission_content + rubric_id)`.
- **Fallback:** If LLM API is down, mark feedback as "pending" and retry via Celery with exponential backoff.

12. DevOps & Deployment

12.1 Docker Compose Services

Production docker-compose.yml with 6 services: api, worker, beat, db, redis, nginx. Each service has health checks, restart policies, and resource limits.

12.2 Nginx Configuration

- **Reverse proxy:** Routes /api/* to FastAPI upstream (port 8000).
- **WebSocket:** Routes /ws/* with Upgrade and Connection headers for WebSocket support.
- **Static files:** Serves React build from /usr/share/nginx/html.
- **File uploads:** client_max_body_size 10M for assignment file uploads.
- **Rate limiting:** limit_req_zone on /api/auth/* (10 requests/minute per IP).

12.3 Deployment Target

- **Development:** Docker Compose on local machine. Hot reload enabled for FastAPI and React.
- **Production:** Render (free tier for demo) or AWS EC2 (t3.small). Single VPS running all 6 containers.
- **Database:** Render PostgreSQL (free tier) or AWS RDS. Alembic for migrations.
- **File storage:** Local filesystem in dev, AWS S3 bucket in production.

12.4 Environment Variables

All configuration via .env file: DATABASE_URL, REDIS_URL, JWT_SECRET, LLM_API_KEY, LLM_MODEL, AWS_S3_BUCKET, CORS_ORIGINS. Never hardcoded.

13. Non-Functional Requirements

Requirement	Target
API Response Time	< 200ms for CRUD operations, < 500ms for dashboard aggregations
AI Feedback Time	< 30 seconds from draft submission to feedback visible
Concurrent Users	Support 200 concurrent WebSocket connections per classroom
Uptime	99.5% (allows ~3.6 hours downtime per month)
Security	JWT with refresh tokens, bcrypt password hashing, CORS, HTTPS, input sanitization
Data Integrity	All submissions immutable after final submit. Append-only audit log for grades.
Scalability	PostgreSQL connection pooling (20 connections). Redis for hot data. Celery workers horizontally scalable.

14. MVP Scope & Phased Roadmap

Phase 1: Core MVP (Weeks 1-4)

The minimum viable product that demonstrates the core innovation.

- Auth (register, login, JWT)
- Classroom CRUD + join via code
- Assignment CRUD with rubric builder
- Draft submission + AI feedback (core innovation)
- Final submission with deadline enforcement
- Basic grading with rubric click-scoring
- Student dashboard with deadline calendar
- Docker Compose with all 6 services

Phase 2: Real-Time & Analytics (Weeks 5-6)

- WebSocket notifications (new assignment, grade released, feedback ready)
- Deadline reminder scheduler (Celery Beat)
- Submission tracker (live count for teachers)
- Grade trends chart (student view)
- Class analytics (teacher view)

Phase 3: Advanced Features (Weeks 7-8)

- Similarity detection (TF-IDF cosine similarity)
- Bulk feedback tool for teachers
- At-risk student detection
- PDF report generation
- Admin/HOD dashboard
- Production deployment to AWS/Render

End of Document